

Dual Position Pure Algorithm: A Cyclic Ring Topology for Rapid Permutation Generation

Yusheng HU^{1*}

^{1*}Independent Researcher, Shangdi East Road, Beijing, 100085, China.

Corresponding author(s). E-mail(s): dr.huyusheng@gmail.com;

Abstract

Generating n -order permutations is a core problem in combinatorial computing. Traditional algorithms rely on local element swaps, where the control overhead increases proportionally with the factorial scale. This paper proposes a universal method to reduce control overhead by constructing a cyclic topology within the context of "in-place generation" for full permutations. Research demonstrates that under the constraint of in-place generation, the high-order state space of n -order permutations can be decoupled and mapped onto a lower-order $(n - 1)$ -dimensional subspace. By utilizing a dual-ring topology design, the algorithm can construct a complete n -order permutation directly in memory with a control overhead proportional to $2 \times (n - 1)$. This method is straightforward and effective, achieving cross-dimensional reduction of computational overhead and providing a general theoretical framework for the structured generation of high-performance combinatorial objects.

Keywords: Permutation Generation; In-place Algorithm; Dual-Ring Topology; Combinatorial Computing; Computational Complexity

MSC Classification: 68Q25 , 05A05

1 Introduction

1.1 Research Background and Significance

The generation and enumeration of full permutations constitute a fundamental challenge in combinatorial computing, with extensive applications spanning graph algorithms, cryptographic design, combinatorial optimization, and high-performance parallel simulation. In practical engineering, permutation generation serves as a critical

data engine for high-performance computing systems, where its efficiency often dictates the performance ceiling of upper-level applications. Given that the solution space grows factorially at $O(n!)$ as the problem size n increases, designing hardware-efficient generation methods capable of enabling low-cost, high-speed enumeration over massive state spaces carries profound theoretical significance for discrete mathematics and essential practical implications for industrial engineering.

1.2 Paradigm Shift: From Swapping to Construction

In-place generation has long been the dominant paradigm for permutation algorithms. Classic methods, such as Heap’s algorithm [3] and the Steinhaus-Johnson-Trotter (SJT) algorithm [5], rely on local element swapping within a one-dimensional array to traverse the entire factorial solution space. However, optimizations for this swapping-based scheme have reached a bottleneck on modern processor architectures, with little progress made over the past six decades.

Recently, research in permutation generation has begun to shift from traditional in-place iteration toward structured construction. The Ring-based Cyclic Permutation Algorithm (RCPA) [7], for instance, leverages an innovative topological structure to implement global construction in non-in-place settings, achieving remarkable gains in generation efficiency. It demonstrates that by modestly relaxing strict in-place constraints and introducing limited auxiliary space, one can effectively decouple states and eliminate excessive control overhead. Drawing inspiration from these insights, this work departs from traditional design philosophies. By adopting a structured construction approach characterized by a global cyclic topology while strictly adhering to in-place requirements, we significantly reduce control overhead, thereby establishing a new trajectory for optimizing in-place permutation generation.

1.3 Core Contributions

Traditional in-place permutation generation suffers from rapidly increasing control overhead as the problem scale grows factorially. To address this bottleneck, we propose a cyclic order-reduction construction scheme for in-place generation, drawing on the factorial number system of the Position Method [6]. Our core contributions are outlined below:

1. **Theory of Spatial Decoupling and Order Reduction for In-place Generation:** We formalize the state space construction of n -order permutations under in-place constraints. This approach eliminates full iteration of the n -order master control flow and achieves universal decoupling and mapping to $(n - 1)$ -dimensional subspaces, providing theoretical support for control overhead reduction.
2. **Efficient Dual-Ring Topological Memory Structure Design:** Two independent $(n - 1)$ -order permutation subspaces are linked via a central element to implement batch generation of n -order permutations. The control iteration overhead is bounded by $2 \times (n - 1)$, overturning the conventional view that overhead must scale with factorial complexity.

3. **Rigorous Proof of Injectivity and Completeness:** Mathematical proofs are presented to confirm that the cyclic concatenation topology produces the complete set of $n!$ permutations with neither duplicates nor missing cases. A universal theoretical framework is thus established for the proposed method.

2 Related Work & Position Method Background

2.1 Overview of Traditional Permutation Algorithms and Cyclic Concepts

Conventional in-place permutation generation algorithms are predominantly built upon local element swapping. Heap’s algorithm [3] employs specific element-pairing rules to traverse the solution space with minimal swaps, whereas the Steinhaus-Johnson-Trotter (SJT) algorithm [5] utilizes element directions to restrict swaps exclusively to adjacent positions. Despite their algorithmic elegance, these approaches involve intensive counter updates, boundary checks, and deep recursion in their inner loops. Consequently, the control overhead scales in proportion to the factorial size of the solution space ($O(n!)$), which severely hampers instruction-level parallelism and degrades performance under the speculative, out-of-order execution paradigm of modern processor architectures.

To mitigate the limitations of sequential local swaps, researchers have explored cyclic generation methods derived from circular shifts and rotations, notably the extensions of Langdon’s circular rotation theory [4]. While these techniques enhance throughput by executing global element movements via specialized hardware (e.g., shift registers) or optimized software designs, they have yet to achieve a fundamental reduction in control overhead across dimensions. The core deficiency lies in the absence of a holistic topological mapping architecture for large-scale execution. To ensure the generation of the full set of $n!$ permutations without duplication or omission, these methods still rely on complex nested counters and multi-way conditional branches to manage rotation and sub-ring configurations. As a result, control complexity remains factorial with respect to the permutation order n , introducing significant branch misprediction overhead in in-place implementations. Effectively exploiting cyclic properties through a global topological lens to compress control flow remains an unresolved challenge for contemporary cyclic permutation algorithms.

2.2 Foundations of the Position Method

The Position Method and its evolved Position Pure (PP) algorithm [6] offer a structured construction paradigm that departs from conventional swap-based permutation generation. Unlike traditional algorithms relying on local state transitions, the PP algorithm maps position indices directly to permutation configurations, treating generation as an incremental transformation within a factorial number system. This structure partially decouples solution space traversal from element relocation, thereby mitigating the overhead of frequent swaps.

Despite these efficiency gains, existing PP algorithms remain constrained by a vertical, hierarchical iteration paradigm. However, the underlying factorial number

system inherently supports global cyclic topologies. By integrating these cyclic properties with the PP mapping mechanism, we can achieve cross-dimensional control overhead reduction even under strict in-place constraints? paving a novel pathway for further performance breakthroughs.

3 Theoretical Framework & Ring Construction

3.1 Universal Decoupling of High-Order State Space

The fundamental defect of traditional permutation generation algorithms lies in the fact that the traversal control of the n -order state space is entirely dependent on an n -order master loop, causing the control flow overhead to be locked into a fate of linear synchronous growth with $n!$. To break this limitation under the constraints of in-place generation, this paper proposes a theory of universal decoupling for high-order state spaces.

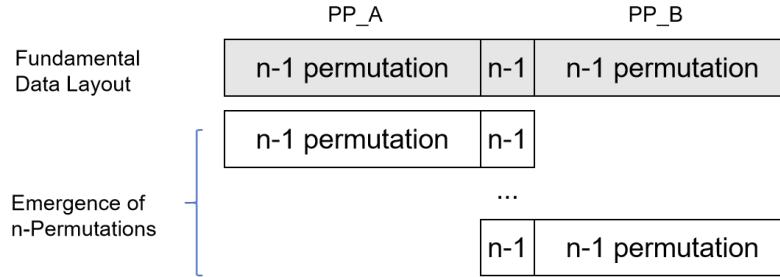


Fig. 1 Dual Position Pure algorithm Enhancement (0-based)

The core essence of this theory is to extend and upscale the traversal logic of low-dimensional subspaces through recursive superposition and displacement mapping of the underlying $(n - 1)$ -order permutation structures, thereby constructing a complete n -order state space from the bottom up. Under the stringent spatial constraints of in-place generation, there is no need to explicitly maintain the state transition trajectory of the entire n -order space. Instead, by designing a lateral topological correlation mechanism, the base-state transitions of low-order subspaces can be used directly as the driving force for high-order permutation generation. This decoupling design, characterized by "control pressure concentrated in low dimensions, with results emerging in high dimensions," provides a solid theoretical basis for fundamentally reducing the amortized control flow overhead of the algorithm.

3.2 Dual-Ring Topological Design and Overhead Analysis

Building upon the spatial decoupling theory, we propose a Dual-Ring Topological Memory Structure to facilitate efficient in-place permutation generation.

3.2.1 Topological Architecture

The memory structure is defined by the configuration: $\mathcal{S} = \langle \mathcal{PP}_A, (n-1), \mathcal{PP}_B \rangle$, where two independent $(n-1)$ -order Position Pure (PP) structures, $\mathcal{PP}_A$ and $\mathcal{PP}_B$, are symmetrically coupled via a central bridging element $(n-1)$. This topology establishes a globally closed cyclic observation window. Geometrically, this mapping allows the generation process to treat the memory as a circular array of length n , where each element transitions within the range $[0, n-1]$.

3.2.2 Batch Generation Mechanism

The design enables an emergent "batch generation" property: each state update triggered by the driving subspace ($\mathcal{PP}_A$) automatically induces a corresponding valid configuration in the complementary subspace ($\mathcal{PP}_B$). Consequently, a single incremental update of the $(n-1)$ -order base state facilitates the simultaneous, non-iterative output of n distinct n -order permutations, effectively eliminating redundant branch operations.

3.2.3 Engineering Implementation and Overhead Optimization

To maximize hardware throughput, the implementation adopts a synchronous mirrored-write strategy:

- **Control Flow:** The main loop drives only $\mathcal{PP}_A$ via state increments.
- **Symmetry Exploitation:** $\mathcal{PP}_B$ is updated via a mirrored synchronous write, where the result is computed by applying a fixed physical offset to the state of $\mathcal{PP}_A$.
- **Efficiency:** This approach bypasses nested base calculations and conditional logic for $\mathcal{PP}_B$, restricting the effective control overhead to that of an $(n-1)$ -order structure.

3.2.4 Algorithmic Complexity

The Dual Position Pure (DPP) algorithm leverages the $O(1)$ amortized complexity of the PP driving source. Given that the construction of an n -order permutation set is derived from two $(n-1)$ -order instances, the total time complexity is bounded by $O((n-1)!)$, achieving a fundamental cross-dimensional reduction in computational overhead.

3.3 Closed-Loop Emergence and Injection Proof of Global n -order Results

To ensure the mathematical completeness of the dual-ring topology construction method, this section provides a rigorous mathematical proof that this topological structure can generate the complete set of n -order full permutations without repetition or omission.

Let S_n be the complete set of n -order full permutations, with a total cardinality of $|S_n| = n!$; let S_{n-1} be the state set of the $(n-1)$ -order subspace driven by the dual-ring topology proposed in this paper. The proof is as follows:

1. **Proof of Injectivity (No Repetition):** Let $P_i, P_j \in S_{n-1}$ be two distinct $(n-1)$ -order base states, such that $P_i \neq P_j$. According to the mapping rules of the dual-ring topology, after each base state is concatenated with the central element $n-1$ through cyclic splicing, it independently expands into n distinct n -order permutations via visual field shifting, forming two permutation families, $\mathcal{A}_i$ and $\mathcal{A}_j$.

Since the Position Pure (PP) algorithm maintains strict uniqueness at the $(n-1)$ -order, the relative topological order of elements within the permutation families $\mathcal{A}_i$ and $\mathcal{A}_j$ is determined entirely by the base states P_i and P_j . Because $P_i \neq P_j$, the cyclic orbits corresponding to the two permutation families do not overlap in space. Furthermore, for the n permutations generated within each group, the starting elements are uniquely determined from 0 to $n-1$, and since the leading elements differ, there is no repetition within each group. In summary, distinct subspace states must correspond to distinct n -order full permutation results, ensuring no conflict or repetition within the generated permutation set.

2. **Proof of Surjectivity and Totality:** According to the construction rules of the dual-ring topology, each $(n-1)$ -order base state linearly produces exactly n distinct n -order full permutations. Given that the complete state space size of an $(n-1)$ -order full permutation is $(n-1)!$, the total quantity N_{total} of n -order permutations globally synthesized by this dual-ring topology is:

$$N_{total} = (n-1)! \times n = n!$$

It follows that the total size of the generated set is numerically strictly equal to the complete set of n -order full permutations $|S_n|$, proving that this topological structure covers the entirety of n -order full permutations.

In conclusion, the dual-ring topological mapping establishes a strict bijection between subspace states and the global high-order space, enabling the algorithm to globally synthesize the complete set of $n!$ n -order full permutations without repetition or omission.

4 Algorithm Implementation & General Framework

4.1 Pseudocode of the Algorithm

Based on the order-reduction theory and dual-ring topology design established in Chapter 3, this paper proposes the Dual Position Pure Algorithm (DPP). In its memory layout, the algorithm fixes the key element $n-1$ in the center as a bridge and axis of symmetry. It independently constructs two Position Pure (PP) algorithm data structures with specific storage boundaries on the left and right sides, forming a symmetric and complementary dual-core driving architecture. The core control flow operates entirely within the low-order $(n-1)$ -dimensional subspace. The pseudocode is as follows:

Algorithm 1 Dual Position Pure Algorithm (DPP)

Require: n (permutation order)**Ensure:** $\mathcal{S}_n$ (complete set of n -order full permutations)

- 1: Initialize memory topology; place element $n - 1$ in the center, and initialize structures PP_A and PP_B on both sides.
 - 2: Set structure offset $\text{Offset} \leftarrow n$.
 - 3: **while** PP_A has not finished traversing its $(n - 1)$ -order state space **do**
 - 4: Generate the next $(n - 1)$ -order base permutation state for PP_A .
 - 5: Read state of PP_A , add Offset , and write to memory region PP_B .
 - 6: Output n permutations of length n based on the cyclic window formed by $(PP_A, n - 1, PP_B)$.
 - 7: **end while**
-

Table 1 presents a performance comparison between the PP and DPP algorithms. The disparity in their algorithmic complexity is reflected in the execution time, where the DPP algorithm achieves a speedup of approximately $n/2$ compared to the PP algorithm.

As evidenced by the pseudocode, the DPP algorithm compresses the high-cost control logic of traditional in-place generation entirely into the $(n - 1)$ -order iteration process of PP_A , while state synchronization for PP_B consumes only minimal memory addressing and double-write overhead. This minimalist main loop design is exceptionally friendly to processor instruction caches and branch predictors, effectively reducing pipeline stalls and enhancing execution efficiency.

4.2 Universal Scalability

The structural paradigm proposed in this paper transcends the specific implementation of the Position Pure (PP) algorithm, offering a universal design pattern for in-place permutation generation. By leveraging its abstract topological characteristics, this approach can be systematically extended to conventional in-place algorithms, including variants of Heap's and SJT algorithms.

Any in-place permutation algorithm can adopt this framework to decompose its state transition logic and element control plane into a dual-subspace architecture coupled via a central hub. By implementing symmetric topological decoupling and sliding window mapping, these algorithms can compress control overhead from $O(n!)$ to the scale of $(n - 1)$ -order, while strictly adhering to in-place constraints.

From a broader perspective of discrete structural construction, the core philosophy of driving high-order batch emergence through low-dimensional base states provides a robust foundation for optimizing other high-performance combinatorial generation tasks. This framework thus establishes a scalable methodology for advancing permutation generation efficiency across diverse algorithmic contexts.

5 Experimental Evaluation

To objectively evaluate the performance of the proposed Dual Position Pure (DPP) algorithm, we adopt an experimental methodology characterized by high transparency and rigorous reproducibility. To mitigate potential biases stemming from hardware microarchitecture variations, background system interference, and compiler-specific behaviors, all experiments are conducted within a standardized, automated continuous integration (CI) workflow using GitHub Actions.

The test programs are implemented in standard C++ and compiled with identical performance optimization flags (GCC `-O3`) within a cloud-virtualized sandbox. By utilizing standardized computing nodes provided by GitHub-hosted runners, we eliminate environmental noise and ensure uniformity across configurations. This rigorous benchmark framework ensures that the reported performance metrics accurately reflect the algorithmic efficiency of the DPP approach relative to traditional methods.

Table 1 Performance comparison between PP and DPP algorithms

n	PP (s)	DPP (s)	Speedup
9	0.000632	0.000133	4.75
10	0.006454	0.001168	5.53
11	0.064187	0.011301	5.68
12	0.707448	0.118985	5.95
13	8.450858	1.403019	6.02

Table 1 presents a performance comparison between the PP and DPP algorithms. The disparity in their algorithmic complexity is reflected in the execution time, where the DPP algorithm achieves a speedup of approximately $n/2$ compared to the PP algorithm.

6 Conclusion & Future Work

With the continuous rise in requirements for the enumeration efficiency of discrete mathematical objects in high-performance computing, research into permutation generation algorithms is entering a brand-new phase of development. For a long time, both academia and industry have been constrained by the conservative paradigm of one-dimensional, in-place local swapping, which has prevented the control flow overhead of permutation generation from escaping the limitation of synchronous growth with the factorial scale, making it difficult to meet the demands of modern high-throughput computing. The research in this paper demonstrates that holistic "structured construction" is not only effective in non-in-place generation scenarios but also possesses immense potential for evolution and performance improvement under the more stringent constraints of in-place generation.

Taking the Position Pure (PP) algorithm, characterized by its factorial number system structure, as the theoretical foundation, this paper transcends the limitations of traditional single-dimensional iteration and successfully constructs a dual-ring memory topological data structure based on low-order subspace driving and central element bridging. Through theoretical derivation and rigorous mathematical proof, it has been verified that this method enables batch mapping and cross-dimensional order-reduction generation of high-order full permutations. It successfully suppresses the control complexity of the algorithm’s main loop to a low-order level, fundamentally overcoming the underlying microarchitectural bottlenecks of traditional swapping algorithms, such as high branch misprediction rates and dense instruction dependencies. These research findings not only enrich the theoretical framework of structured generation for combinatorial objects but also provide a novel problem-solving paradigm and technical path for exploring high-throughput underlying combinatorial data sources.

References

- [1] Robert Sedgewick. Permutation generation methods. *Computing Surveys*, 9:137–164, 1977.
- [2] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 2: Generating All Tuples and Permutations*. Addison-Wesley Professional, Reading, MA, 2005.
- [3] B. R. Heap. Permutations by interchanges. *The Computer Journal*, 6(3):293–294, 1963.
- [4] G. G. Langdon. Rotation-based permutation generation methods. *IBM Journal of Research and Development*, 17(5):420–425, 1973.
- [5] S. M. Johnson. “Generation of permutations by adjacent transpositions,” *Mathematics of Computation*, vol. 17, no. 83, pp. 282–285, 1963.
- [6] Yusheng Hu. Position Method: A Linear-Time Generation Algorithm for Permutations. *SSRN*, February 22, 2026. Available at <https://ssrn.com/abstract=6285581> or <http://dx.doi.org/10.2139/ssrn.6285581>.
- [7] Yusheng Hu. Ring-Cascade: A Sub-constant Amortized Time Algorithm for High-Speed Permutation Generation. *SSRN*, April 09, 2026. Available at <https://ssrn.com/abstract=6553539> or <http://dx.doi.org/10.2139/ssrn.6553539>.